# PhysioSense: A Distributed Platform for AI-Assisted Remote Physiotherapy Monitoring and Clinical Decision Support

Ronak Salekar[a] ,Harshali Dhindale[b] , Aayush Meshram[c] , Sarang Khutle[d]

[abcd]Department of Computer Engineering, TPCT's Terna Engineering College, Nerul, Navi Mumbai, 400706

ronaksalekar2324@ternaengg.ac.in | harshalidhindale2324@ternaengg.ac.in | aayushmeshram2324@ternaengg.ac.in | sarangkhutle2324@ternaengg.ac.in

## Abstract

Adherence to clinician-prescribed home exercise programmes remains one of the most persistent barriers to effective musculoskeletal rehabilitation. Existing solutions either provide passive instructional video content with no objective movement assessment, or require expensive depth-sensing hardware that is inaccessible for most outpatient settings. This paper presents PhysioSense, a distributed, open-architecture platform that couples a React Native mobile patient application with a server-side Python AI vision microservice to deliver real-time biomechanical feedback through a standard smartphone camera. The platform implements six exercise-specific analysers (squat, bicep curl, shoulder raise, lunge, knee extension, and hamstring curl), a six-state deterministic finite-state machine for hysteresis-resistant repetition counting, and a geometric compensation-detection layer that identifies three clinical movement patterns: knee valgus, excessive trunk lean, and torso sway. Clinician-prescribed range-of-motion parameters are propagated from a Spring Boot REST backend through a bidirectional WebSocket channel, enabling per-patient treatment-aware validation in real time. A React 19 clinical dashboard provides physiotherapists with asynchronous session review, and a Java Spring Boot CLI implements role-segmented administrative provisioning without direct database access. We describe the system architecture, the AI pipeline in mathematical detail, the verified technology stack, and the complete WebSocket message protocol as determined from source-code inspection. We discuss the inherent limitations of monocular 2D pose estimation for biomechanical assessment, enumerate open questions that require prospective clinical validation, and identify directions for future research.

**Keywords**: telerehabilitation, pose estimation, MediaPipe, joint angle calculation, exercise monitoring, clinical decision support, finite-state machine, remote physiotherapy, React Native, FastAPI

## 1.Introduction

The global burden of musculoskeletal conditions—which include osteoarthritis, post-operative rehabilitation, chronic low back pain, and post-stroke motor recovery—places enormous demand on physiotherapy services. Systematic reviews consistently report that adherence to home exercise programmes (HEPs) is the single most modifiable predictor of recovery outcomes, yet rates of non-adherence in the literature range from 30% to 70% [Jack et al., 2010; Peek et al., 2020]. Conventional HEP delivery relies on written instruction sheets and periodic face-to-face clinic visits, providing no objective movement assessment between appointments and leaving the treating clinician unable to distinguish patients who completed their exercises correctly from those who compensated or avoided particular movement ranges.

Two technological trajectories have emerged to address this gap. The first is wearable sensing: inertial measurement units (IMUs) attached to body segments provide high-frequency kinematic data and have demonstrated clinical-grade accuracy for range-of-motion (ROM) measurement, but require hardware procurement, device pairing, and patient compliance with donning and doffing procedures. The second is computer-vision-based markerless pose estimation: commodity RGB cameras, including those already present in patients' smartphones, are used to estimate body joint positions from video frames without any additional sensor. This approach eliminates hardware cost and significantly reduces setup friction, making it attractive for large-scale telerehabilitation deployment.

The publication of Google's BlazePose landmark detection architecture and its subsequent integration

into the MediaPipe framework [Bazarevsky et al., 2020; Lugaresi et al., 2019] lowered the barrier to smartphone-based markerless pose estimation substantially. Clinical validation studies comparing MediaPipe landmark-derived joint angles against optical motion capture and goniometry have reported root-mean-square errors in the range of 2.1° to 5.8° for major joint angles under controlled conditions [Khurana et al., 2023; Colyer et al., 2018], a margin that is arguably acceptable for many physiotherapy monitoring tasks where the goal is detection of substantial movement deviation rather than millimetre-precision kinematics.

Despite this evidence base, existing telerehabilitation platforms typically address only one dimension of the clinical workflow. Some provide exercise prescription and scheduling without any movement analysis. Others implement movement classification without connecting it to clinician-issued prescriptions. Few integrate real-time feedback, asynchronous clinical review, administrative provisioning, and treatment-aware movement validation within a single coherent architecture.

This paper describes **PhysioSense**, a full-stack distributed platform designed to close this integration gap. Our primary contributions are:

1. A verified, end-to-end software architecture integrating a React Native mobile application, a Python FastAPI AI vision microservice, a Java Spring Boot backend, a React clinical dashboard, and an administrative CLI, with all communication interfaces documented from source-code inspection.
2. A treatment-parameterised AI pipeline in which clinician-prescribed target and allowed range-of-motion boundaries are transmitted from the backend to the AI service at session initialisation, enabling per-patient movement validation rather than generic population-level thresholds.
3. A six-state deterministic finite-state machine with exercise-specific hysteresis and minimum duration thresholds that prevents noise-induced false-positive repetition counts.
4. A three-flag compensation detection layer that quantifies relative landmark geometry to identify knee valgus, excessive trunk lean, and torso sway during exercise execution.
5. A comprehensive clinical session summary schema that aggregates repetition-level form quality, average peak ROM, range-violation counts, and compensation observations into a structured record visible to the treating physiotherapist.

We do not present PhysioSense as a validated clinical device. No controlled clinical trials measuring patient outcomes are reported. All angle thresholds are programmatic implementation constants, not clinically validated normative data. The system is described as an engineered research artefact suitable for prospective evaluation in clinical research settings.

## 2. Background and Related Work

### 2.1 Telerehabilitation: Evidence and Limitations

Systematic reviews of telerehabilitation across musculoskeletal conditions generally report clinical outcomes that are non-inferior to conventional outpatient care [Cottrell et al., 2017; Maddison et al., 2019]. Smartphone-delivered HEP platforms with video instruction and push-notification reminders achieve adherence rates of 80%–90% over short-to-medium intervention windows, though adherence decay beyond eight weeks is well documented without supplementary therapist interaction [Bandura, 1997; Peek et al., 2020]. None of these positive adherence findings can be attributed to real-time movement quality feedback specifically; most evaluated platforms provide scheduling and video instruction only.

### 2.2 Markerless Pose Estimation for Rehabilitation

OpenPose [Cao et al., 2017] was among the first widely adopted multi-person pose estimation frameworks, but its computational requirements precluded mobile deployment. BlazePose [Bazarevsky et al., 2020] introduced a two-stage detector-regressor pipeline that achieves real-time inference (≥30 fps) on mid-range smartphones, making it the de-facto baseline for mobile physiotherapy applications. The MediaPipe PoseLandmarker API [Lugaresi et al., 2019] wraps BlazePose behind a unified task-file interface and exposes 33 body landmarks with normalised image-plane coordinates and a per-landmark visibility score.

Recent validation studies establish that MediaPipe landmark-derived 2D joint angles are most accurate when the exercise movement occurs predominantly in the camera plane, i.e., when the patient is correctly aligned as a profile view for sagittal-plane exercises or a frontal view for frontal-plane exercises [Khurana et al., 2023]. Oblique camera angles introduce cosine projection error. This is a fundamental limitation of monocular 2D methods that cannot be overcome without stereo camera rigs or depth sensors.

### 2.3 Automated Repetition Counting

Early repetition-counting systems used threshold-crossing on angle time-series: a repetition was counted each time the angle crossed a fixed boundary. This approach is vulnerable to oscillation around the threshold ("Zeno effect"), producing falsely inflated counts. Subsequent approaches applied hysteresis: separate entry and exit thresholds separated by a dead-band. The finite-state machine (FSM) formalism provides a principled framework for multi-state hysteresis that includes not only a PEAK state but also a RETURNING state, ensuring the full movement cycle is completed before a repetition is recorded [Shotton et al., 2011]. Machine learning approaches using recurrent networks or hidden Markov models can learn temporal dynamics directly from data [Lea et al., 2016] but require labelled training corpora and lack the transparency of rule-based FSMs for clinical audit purposes.

### 2.4 Clinical Decision Support in Rehabilitation

Clinical Decision Support Systems (CDSS) in rehabilitation are characterised by their integration of patient-specific prescription parameters with real-time measurement [Shortliffe & Cimino, 2014]. The key distinction between a simple pose-estimation application and a CDSS is whether the system validates movement against the individual patient's prescribed therapeutic ROM rather than against generic population norms. Treatment-aware validation requires a communication channel between the clinical record system and the AI inference layer. Existing literature documents few systems that have implemented this channel at the level of individual session initialisation.

## 3. System Architecture

### 3.1 Overview

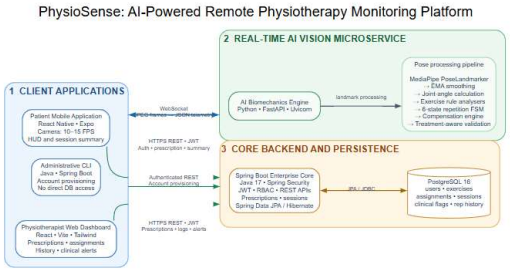



*Fig. 1*

PhysioSense comprises five distinct software subsystems deployed as separate processes with well-defined communication interfaces. The verified technology stack, established from package.json, pom.xml, requirements.txt, and ARCHITECTURE.md, is presented in Table 1.

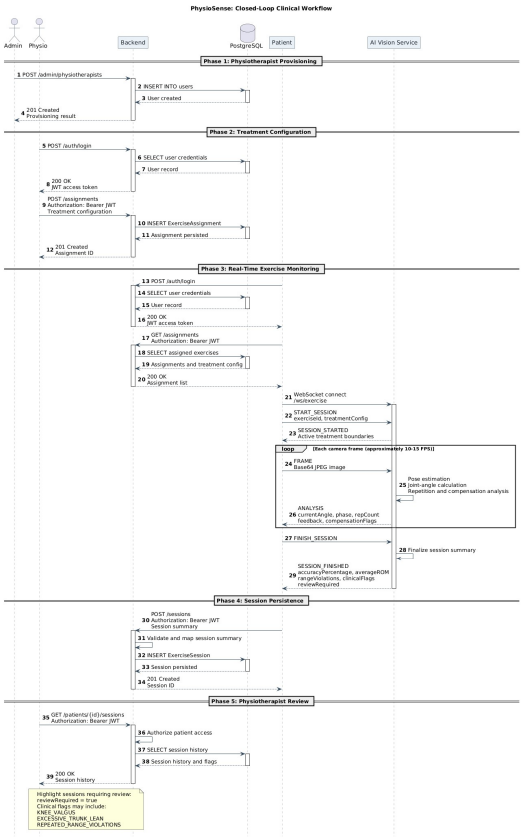
**Table 1: Verified Technology Stack**

| Subsystem | Technology | Confirmed Versions |
|---|---|---|
| Patient mobile application | React Native, Expo | React Native 0.86.3, Expo 57.0.26 |
| Mobile routing | Expo Router, React Navigation | Expo Router 57.0.24 |
| Mobile styling | NativeWind | 4.1.23 |
| Mobile HTTP client | Axios | 1.8.4 |
| Clinical web dashboard | React, Vite | React 19.2.8, Vite 8.3.0 |
| Dashboard routing | React Router DOM | 7.18.4 |
| Dashboard styling | Tailwind CSS | 4.3.3 |
| Backend application server | Java 17, Spring Boot | Spring Boot 4.1.1 |
| Backend ORM | Spring Data JPA, Hibernate | via Spring Boot 4.1.1 |
| Backend authentication | Spring Security, JJWT | JJWT 0.12.6 |
| Backend database | PostgreSQL | Schema via Hibernate DDL |
| AI vision microservice | Python 3.12, FastAPI | FastAPI ≥0.115.0 |
| AI ASGI server | Uvicorn | ≥0.30.0 |
| AI pose model | MediaPipe PoseLandmarker Lite | MediaPipe 0.10.35 |
| AI image processing | OpenCV (contrib) | ≥4.10.0 |
| AI numerical computation | NumPy | ≥1.26.0 |
| AI data validation | Pydantic | ≥2.8.0 |
| Administrative CLI | Java 17, Spring Boot | Spring Boot 4.1.1 |

### 3.2 Communication Topology

The system employs two transport protocols serving distinct purposes:

**WebSocket (ws://)** — Used exclusively for the real-time bidirectional channel between the patient mobile application and the AI vision microservice. Camera frames (base64-encoded JPEG strings) travel client-to-server; structured JSON telemetry messages travel server-to-client. This channel accepts both JSON text messages with a "image" field and raw binary WebSocket messages containing JPEG bytes directly (both paths confirmed in exercise_socket.py, lines 44–48 and 143–146). The WebSocket endpoint is at ws://<host>:8000/ws/exercise. In the current implementation the WebSocket channel does not




require JWT authentication; patient identity is conveyed through the START_SESSION JSON payload.

**HTTP REST** — Used for all other inter-component communication: mobile application authenticates against and submits session summaries to the Spring Boot backend; the clinical dashboard reads patients, assignments, and sessions; the administrative CLI provisions physiotherapist accounts. All protected REST endpoints require a JWT Bearer token in the Authorization header ,enforced by Jwt AuthenticationFilter.

**Spring Data JPA → PostgreSQL** — The Spring Boot backend persists all domain entities. The schema is managed through Hibernate's ddl-auto=update; no explicit SQL migration files are present in the repository.

The AI microservice does not communicate with PostgreSQL. Session summary data flows: AI service → (WebSocket) → mobile app → (REST POST) → Spring Boot backend → PostgreSQL.

### 3.3 Backend Domain Model

The Spring Boot backend defines the following JPA entities (all verified from source files):

- **users** (User.java): id, name, email, password (BCrypt-hashed), role (string values: PATIENT, PHYSIOTHERAPIST, ADMIN).
- **exercises** (Exercise.java): id, name, description, bodyPart, difficulty, durationSeconds, videoUrl, exerciseCode.
- **exercise_assignments** (ExerciseAssignment.java): FK patient, FK physiotherapist, FK exercise; prescription fields: sets, repetitions, frequency, startDate, endDate, instructions; clinical treatment parameters: targetMinAngle, targetMaxAngle, allowedMinAngle, allowedMaxAngle, side, monitoringLevel, restrictions.
- **exercise_sessions** (ExerciseSession.java): FK assignment, FK patient, FK exercise; exerciseCode, completedReps, goodReps, accuracyPercentage, durationSeconds, startedAt, completedAt; repHistory (TEXT — JSON-serialised array); clinical observations: averageRom, poseConfidence, rangeViolations, incompleteReps, reviewRequired, clinicalFlags (TEXT), treatmentFeedback.

**Note:** repHistory and clinicalFlags are stored as flat TEXT columns containing JSON, preventing SQL-level querying of individual repetition attributes. This is a design trade-off that simplifies persistence but limits backend analytics.

### 3.4 Role-Based Access Control

JwtAuthenticationFilter extracts a role claim from the JWT and constructs a SimpleGrantedAuthority ("ROLE_" + role). Three roles are enforced: ROLE_PATIENT, ROLE_PHYSIOTHERAPIST, and ROLE_ADMIN. The administrative CLI operates exclusively through authenticated REST calls, holding no direct database access.

**Security note:** app.jwt.expiration-ms=0 in application.properties means JWT tokens do not expire in the current configuration. This requires correction before production deployment.

## 4. AI Vision Pipeline

The AI vision pipeline executes ten sequential steps per camera frame within ExerciseSessionManager .process_frame (verified from session/manager.py):

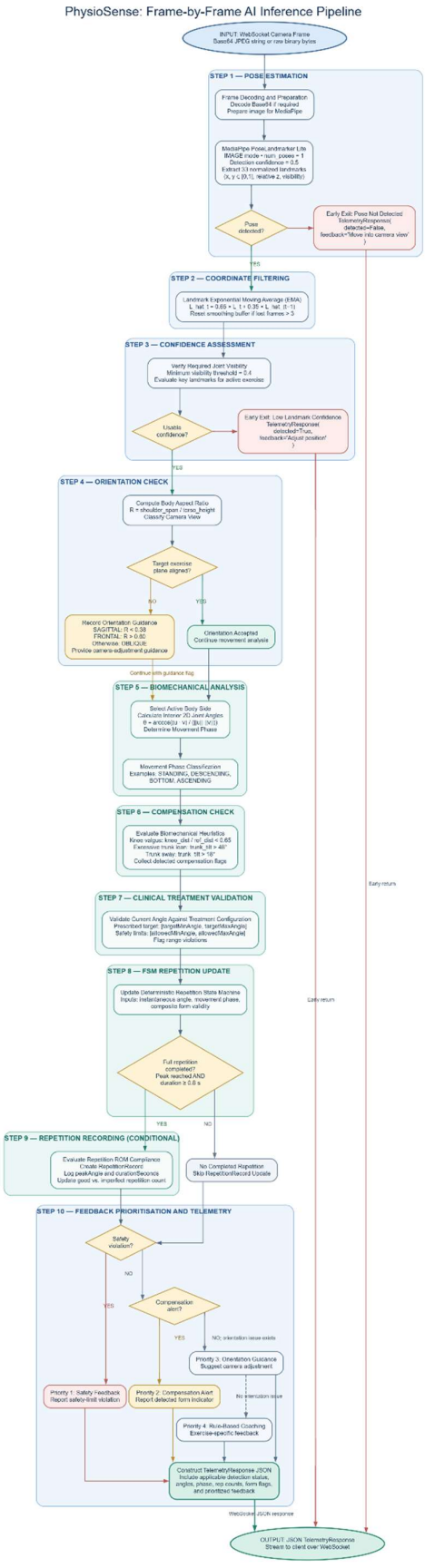
1. Pose estimation from image input
2. EMA landmark smoothing
3. Pose confidence evaluation
4. Camera orientation assessment
5. Exercise-specific biomechanical analysis
6. Compensation detection
7. Treatment-aware angle validation
8. FSM repetition state update
9. Repetition recording (conditional on state transition)
10. Feedback prioritisation and telemetry serialisation

### 4.1 Frame Acquisition and Encoding

The patient mobile application captures frames using expo-camera (v57.0.6). Captured frames are base64-encoded as JPEG strings and serialised in a JSON FRAME message (verified from visionProtocol.js, createFrameMessage):

json

```
{
  "type": "FRAME",
  "image": "<base64-encoded JPEG>",
  "timestamp": 1730118200000,
  "sequence": 42}
```



The client documentation targets 10–15 FPS, though no client-side frame throttle is implemented in the hook; actual rate depends on device capture capability and network latency. The server-side PoseEstimator.estimate_from_base64 strips data URI prefixes () before decoding. OpenCV's cv2.imdecode with IMREAD_COLOR decodes JPEG bytes to a BGR NumPy array; cv2.cvtColor(bgr, cv2.COLOR_BGR2RGB) converts to RGB for MediaPipe input. No image rescaling is performed. Transmitted frame resolution depends on the Expo Camera capture configuration and is not fixed in the current implementation.

### 4.2 Landmark Smoothing

Before geometric computation, raw landmark coordinates are filtered by an exponential moving average (EMA) in ExerciseSessionManager._smooth_landmarks (lines 102–116 of manager.py). With smoothing coefficient α = 0.65:

$$\hat{L}_t(j) = 0.65 \cdot L_t(j) + 0.35 \cdot \hat{L}_{t-1}(j)$$

where $L_t(j)$is the raw coordinate of landmark $j$at frame $t$. If more than three consecutive frames fail pose detection (_lost_frame_count > 3), the smoothing buffer is cleared to prevent stale carry-forward.

### 4.3 Human Pose Estimation

The system uses the **MediaPipe PoseLandmarker Lite** model (pose_landmarker_lite.task), confirmed from the model URL in pose_estimator.py. Initialisation parameters (verified from _init_detector):

| Parameter | Value |
|---|---|
| Running mode | IMAGE (per-frame, no temporal tracking) |
| num_poses | 1 |
| min_pose_detection_confidence | 0.5 |
| min_pose_presence_confidence | 0.5 |
| min_tracking_confidence | 0.5 |
| output_segmentation_masks | False |

The IMAGE running mode means MediaPipe's internal temporal smoothing (available in VIDEO mode) is not engaged. The output is 33 NormalizedLandmark objects per pose, each with x, y (normalised to image dimensions in [0, 1]), z (estimated relative depth, not metrically calibrated), and visibility (model-confidence score in [0, 1]).

normalize_landmarks (pose/landmarks.py) converts the list to a camelCase-keyed dictionary (e.g., "leftKnee", "rightHip") and appends two virtual landmarks computed as coordinate arithmetic means: "midShoulder" and "midHip".

**Known limitations of this configuration:**

- The Lite model offers lower landmark accuracy than the Full or Heavy variants.
- IMAGE mode disables MediaPipe's internal temporal smoothing; only application-level EMA (α = 0.65) is applied.
- x and y coordinates are normalised image-plane values; z is a monocular depth estimate with significant uncertainty.
- The visibility score is a model confidence estimate, not a calibrated probability; landmarks may receive high visibility despite partial occlusion through model extrapolation.

### 4.4 Joint Angle Calculation

The calculate_angle_3points function (geometry/angles.py) computes the interior angle at vertex B given three landmarks A, B, C:

$$\vec{u} = A - B,\ \vec{v} = C - B$$

$$\theta = \arccos\left(\frac{\vec{u} \cdot \vec{v}}{\| \vec{u} \| \| \vec{v} \|}\right)$$

The cosine is clamped to $[-, 11]$before inversion to prevent floating-point domain errors. If either vector magnitude is $< 10^{-6}$, the function returns 0.0 (degenerate guard). The result is converted to degrees and rounded to one decimal place.

Two auxiliary angle functions are also implemented:

- calculate_angle_with_vertical(p_start, p_end): angle between vector $(p_{end} - p_{start})$and the downward vertical $(0, 1)$. Returns ≈0° for a hanging arm, ≈90° for a horizontal arm, ≈180° for an overhead arm.
- calculate_angle_with_horizontal(p_start, p_end): angle relative to the horizontal vector $(1, 0)$.

All calculations operate exclusively on **2D normalised image-plane coordinates** (x, y only). The vector_2d function in geometry/vectors.py

extracts only these two components; z is not used in any angle computation.

These angle calculations produce geometric estimates of apparent joint angle in the image plane. They do not constitute clinically validated goniometric measurements. Their accuracy is contingent on patient alignment with the exercise-appropriate camera plane.

### 4.5 Exercise-Specific Analysers

Six exercise analysers are implemented as concrete subclasses of BaseExerciseAnalyzer (exercises/ base.py). Each returns an ExerciseFrameResult with fields: current_angle, phase, is_good_form, feedback , form_metrics. Table 2 summarises all six analysers.

**Table 2: Exercise Analyser Configuration (all values verified from source)**

| Exercise | File | Primary Landmarks | Angle Metric | Phase Boundaries | Peak Condition | Comp. Checks |
|---|---|---|---|---|---|---|
| Squat | squat.py | hip–knee–ankle (bilateral or best side) | Interior knee angle | STANDING ≥155°, BOTTOM ≤110° | ≤110° | Knee valgus; trunk lean >48° |
| Bicep Curl | bicep_curl.py | shoulder–elbow–wrist (active side) | Interior elbow angle | EXTENDED ≥145°, CONTRACTED ≤65° | ≤65° | Trunk sway >18°; elbow drift >0.35 |
| Shoulder Raise | shoulder_raise.py | shoulder→elbow vector from vertical | Elevation from vertical | DOWN ≤30°, TARGET 75°–105° | ≥75° | Trunk sway >18° |
| Lunge | lunge.py | hip–knee–ankle (lead knee) | Interior knee angle | STANDING ≥145°, BOTTOM ≤100° | ≤100° | Knee valgus; trunk lean >48°; knee/ankle offset >0.22 |
| Knee Extension | knee_extension.py | hip–knee–ankle (extending side) | Interior knee angle | BENT ≤95°, FULL_EXTENSION ≥160° | ≥160° | Trunk sway >18° |
| Hamstring Curl | hamstring_curl.py | hip–knee–ankle (curling side) | Interior knee angle | EXTENDED ≥150°, CONTRACTED ≤85° | ≤85° | None specific |

**Bilateral side-selection logic (verified per analyser):**

- *Squat:* mean visibility of (hip, knee, ankle) per side compared; if difference < 0.2, angles are averaged; otherwise higher-visibility side used.
- *Bicep curl:* side with angle < 135° is the active side; when both are similar, higher combined (elbow + wrist) visibility wins.
- *Shoulder raise:* side with elevation > 40° is active; when both are similar, higher elevation wins.
- *Lunge:* side with more than 10° lower knee angle is the lead knee; when similar, the lower angle is used.
- *Knee extension:* side with higher angle (more extended) is tracked.
- *Hamstring curl:* side with lower angle (more curled) is tracked.

**Additional form rules:**

- Squat: knee angle < 60° sets is_good_form = False ("too deep"); no penalty for the range 60°–110°.
- Bicep curl: horizontal distance between elbow and ipsilateral hip > 0.35 normalised units sets is_good_form = False (elbow drift).
- Shoulder raise: shoulder–elbow–wrist angle < 120° sets is_good_form = False (bent elbow); elevation > 105° also sets is_good_form = False.
- Lunge: horizontal knee–ankle offset > 0.22 normalised units sets is_good_form = False (knee passing over toes).

### 4.6 Camera Orientation Assessment

evaluate_camera_orientation (geometry/orientation.py) computes the ratio of shoulder horizontal span to torso height in normalised image coordinates:

$$R = \frac{| L_{sh,x} - R_{sh,x} |}{| \text{mid_hip}_y - \text{mid_shoulder}_y |}$$

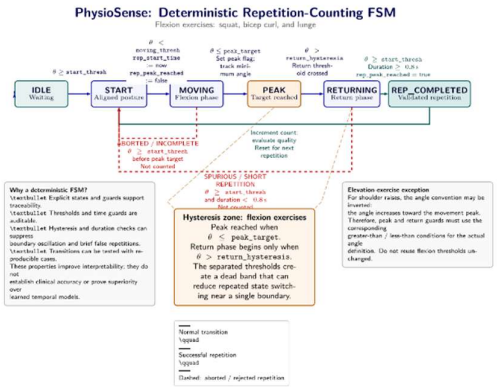
- $R < 0.38$: SAGITTAL (profile) view
- $R > 0.60$: FRONTAL view
- $0.38 \leq R \leq 0.60$: OBLIQUE

Preferred planes (verified from EXERCISE_PREFERRED_PLANES): SAGITTAL for squat, lunge, bicep curl, hamstring curl, knee extension; FRONTAL for shoulder raise. When the observed plane mismatches the required plane, is_optimal_view = False is returned with a

guidance message. Perspective accuracy scores: 1.0 (optimal match), 0.75–0.80 (oblique), 0.60 (full mismatch).

### 4.7 Finite-State Machine for Repetition Counting

ExerciseStateMachine (session/state_machine.py) implements a six-state FSM. States: IDLE, START, MOVING, PEAK, RETURNING, REP_COMPLETED (the last is a transition outcome, not a persistent state; the machine returns to START after crediting a rep).




**Verified default thresholds (configurethresholds):**

| Parameter | SQUAT | LUNGE | BICEP_CURL | HAMSTRING_CURL | KNEE_EXT | SHOULDER_RAISE |
|---|---|---|---|---|---|---|
| start_thresh | 150.0° | 150.0° | 140.0° | 140.0° | 85.0° | 32.0° |
| moving_thresh | 142.0° | 142.0° | 130.0° | 130.0° | 75.0° | 38.0° |
| peak_target | 110.0° | 100.0° | 65.0° | 85.0° | 25.0° | 75.0° |
| return_hysteresis | 118.0° | 108.0° | 78.0° | 98.0° | 37.0° | 68.0° |
| Exercise type | Flexion | Flexion | Flexion | Flexion | Flexion | Elevation |

**Flexion exercise transition rules:**

- IDLE → START: angle ≥ start_thresh
- START → MOVING: angle < moving_thresh (records rep_start_time = now)
- MOVING → PEAK: angle ≤ peak_target (sets rep_peak_reached = True)
- MOVING → START: angle ≥ start_thresh (aborted rep — not counted)
- PEAK → RETURNING: angle > return_hysteresis
- RETURNING → START: angle ≥ start_thresh AND (now - rep_start_time) ≥ 0.8 s AND rep_peak_reached == True → **repetition recorded**

**Elevation exercise (shoulder raise) — inverted transitions:**

- IDLE → START: angle ≤ start_thresh
- START → MOVING: angle > moving_thresh
- MOVING → PEAK: angle ≥ peak_target
- PEAK → RETURNING: angle < return_hysteresis
- RETURNING → START: angle ≤ start_thresh (same duration and peak guards apply)

**Treatment parameterisation:** when a TreatmentConfig is supplied at session initialisation, peak_target is set to cfg.targetMaxAngle (flexion) or cfg.targetMinAngle (elevation), and start_thresh is adjusted accordingly (peak_target + 35° for squat/lunge, peak_target + 13° for curls, peak_target − 18° for shoulder raise).

The min_rep_duration = 0.8 s guard prevents noise-burst false positives. The rep_peak_reached guard ensures the therapeutic target was genuinely reached. Peak angle is tracked as the extremum of all angles within the MOVING and PEAK states and is recorded in repHistory.

The current_rep_good_form flag is initialised to True at START → MOVING transition and set to False if any frame in MOVING, PEAK, or RETURNING reports is_good_form = False. Form quality from compensation checks and treatment violations is OR-ed into this flag before the FSM receives it (manager.py, line 238).

**Why a FSM rather than a learned temporal classifier?**
The FSM implementation is fully transparent: every threshold is an inspectable program constant; every transition is a deterministic Boolean predicate. This makes clinical audit tractable — a false-positive or false-negative repetition count can be traced to a specific comparison. A learned temporal model (e.g., CNN-LSTM) could generalise more flexibly across patients but requires a labelled exercise corpus and produces opaque activation patterns that are harder to audit in a clinical context.

### 4.8 Compensation Detection

detect_compensations (clinical/compensations.py) evaluates three movement compensations using normalised 2D landmark coordinates. These are

**algorithmic geometric indicators**, not clinically validated diagnostic criteria.

**KNEE_VALGUS_DETECTED** (squat and lunge; phases: DESCENDING, BOTTOM, MOVING, PEAK)

$$r_{valgus} = \frac{|\ L_{knee,x} - R_{knee,x}\ |}{\max\left(|, L_{ankle,x} - R_{ankle,x}\ ||\ L_{hi\ ,x} - R_{hi\ ,x}\ |\right)}$$

Triggered when $r_{valgus} < 0.65$AND reference distance $> 0.10$normalised units. Severity: "WARNING".

This heuristic captures inward collapse of knees relative to ankle–hip width—a pattern associated with patellofemoral loading in clinical biomechanics literature. The 2D projection metric may produce false positives when the camera is not frontal.

**EXCESSIVE_TRUNK_LEAN** (squat and lunge)

$$\theta_{trunk} = \arccos\left(\frac{(\text{mid_hip} - \text{mid_shoulder}) \cdot (0,1)}{\|\ \text{mid_hip} - \text{mid_shoulder}\ \|}\right)$$

Triggered when $\theta_{trunk} > 48°$. Severity: "WARNING". A tilt of up to 35° is acknowledged as physiologically normal during deep squat descent. The 48° boundary is a programmatic constant.

**TRUNK_SWAY_COMPENSATION** (shoulder raise, bicep curl, knee extension)

Same $\theta_{trunk}$computation. Triggered when $\theta_{trunk} > 18°$. Captures compensatory backward lean during bicep curls and forward lean during shoulder raises. Severity: "WARNING".

All detected flags are accumulated in compensation_flags_observed (a set) throughout the session, preventing duplicate entries. They are reported in both per-frame telemetry and the session summary.

### 4.9 Treatment-Aware Movement Validation

validate_angle_against_treatment (clinical/movement_constraints.py) compares the current angle against TreatmentConfig boundaries at every active frame:

- **VIOLATION** (flexion, angle < allowedMinAngle): "Exceeding minimum safe angle limit."
- **VIOLATION** (elevation or PEAK phase, angle > allowedMaxAngle): "Exceeded maximum safe range."
- **TARGET_ACHIEVED** (PEAK phase, targetMinAngle ≤ angle ≤ targetMaxAngle).
- **WARNING** (PEAK phase, outside target but within allowed range).
- **VALID**: all other cases.

Two or more VIOLATION events per session appends REPEATED_RANGE_VIOLATION to live_clinical_flags and sets reviewRequired = True.

Per-repetition ROM compliance (evaluate_rep_rom_compliance): the peak angle is compared against target and allowed ranges. Under STANDARD monitoring level, an 8° tolerance band is applied around the target boundaries before the rep is classified as non-compliant.

### 4.10 Pose Confidence Evaluation

evaluate_pose_confidence (pose/confidence.py) returns a scalar confidence score in [0, 1] and a boolean is_usable. The score is the maximum mean visibility across the relevant limb chains (e.g., left leg vs. right leg for SQUAT). If no chain achieves a minimum landmark visibility of 0.4, is_usable = False and the frame is not forwarded to the analyser. Session-average confidence is included in SessionSummary.poseConfidence. If session-average confidence < 0.60, LOW_POSE_CONFIDENCE is raised.

### 4.11 WebSocket Protocol

The bidirectional message protocol is formally defined in visionProtocol.js (client) and schemas/telemetry.py (server). Verified message schemas:

**Client → Server:**

| Type | Key Fields |
|---|---|
| START_SESSION | exerciseCode, targetReps, patientId, treatmentConfig (nested: targetMinAngle, targetMaxAngle, allowedMinAngle, allowedMaxAngle, side, monitoringLevel, instructions, restrictions) |

| Type | Key Fields |
|---|---|
| FRAME | image (base64 JPEG string), sequence (int), timestamp (ms epoch) |
| PAUSE_SESSION | timestamp |
| RESUME_SESSION | timestamp |
| FINISH_SESSION | timestamp |

**Server → Client:**

| Type | Key Fields |
|---|---|
| SESSION_STARTED | exerciseCode, targetReps, targetRange {min, max}, allowedRange {min, max} |
| ANALYSIS | detected (bool), landmarks (dict of LandmarkPoint {x, y, z, visibility}), currentAngle (float), completedReps, goodReps, imperfectReps, currentPhase (string), isGoodForm (bool), feedback (string), confidence, poseConfidence, repHistory (list), exercise, status (VALID/TARGET_ACHIEVED/WARNING/VIOLATION), targetRange, allowedRange, clinicalFlags (list) |
| SESSION_PAUSED | status |
| SESSION_RESUMED | status |
| SESSION_FINISHED | summary (full SessionSummary object — see below) |
| ERROR | error (string) |

The SessionSummary transmitted at session end contains: exerciseCode, completedReps, goodReps, imperfectReps, accuracyPercentage (integer percentage), durationSeconds, repHistory (list of RepetitionRecord objects each with rep, isGoodForm, peakAngle, durationSeconds, feedback), feedbackSummary (natural-language summary string), averageROM, rangeViolations, incompleteReps, poseConfidence, reviewRequired, clinicalFlags, clinicalNotes.

### 4.12 Clinical Safety Evaluation

At session completion, evaluate_session_safety (clinical/safety_rules.py) evaluates eight criteria:

| Clinical Flag | Trigger Condition |
|---|---|
| LOW_POSE_CONFIDENCE | Session-average confidence < 0.60 |
| REPEATED_RANGE_VIOLATION | Range violations ≥ 2 |
| EXCESSIVE_MOVEMENT | Range violations ≥ 1 |
| INSUFFICIENT_MOVEMENT | Average ROM < targetMinAngle − 10° |
| REPEATED_INCOMPLETE_REPS | Imperfect reps ≥ 3 or ≥ 40% of total |
| WRONG_SIDE | Laterality mismatch (externally reported) |
| SESSION_INTERRUPTED | Completed reps < target reps / 2 |
| SUBOPTIMAL_CAMERA_ORIENTATION | Orientation not optimal throughout session |
| KNEE_VALGUS_DETECTED | From compensation detection |
| EXCESSIVE_TRUNK_LEAN | From compensation detection |
| TRUNK_SWAY_COMPENSATION | From compensation detection |

reviewRequired = True is set when any of {REPEATED_RANGE_VIOLATION, INSUFFICIENT_MOVEMENT, REPEATED_INCOMPLETE_REPS, WRONG_SIDE, KNEE_VALGUS_DETECTED, EXCESSIVE_TRUNK_LEAN} is present.

**Feedback prioritisation** (verified from manager.py, lines 263–276): treatment violation feedback > compensation alert message > camera orientation guidance > analyser rule feedback.

## 5.Clinical Dashboard and Administrative CLI

### 5.1 Physiotherapist Clinical Dashboard

The clinical web dashboard (frontend/frontend/src/) is built with React 19.2.8, Vite 8.3.0, React Router DOM 7.18.4, and Tailwind CSS 4.3.3. Pages include: Dashboard, Patients, PatientDetail, Assignments, AssignmentCreate, AssignmentDetail, Exercises, ExerciseCreate, ExerciseDetail, Profile, and Settings. The physiotherapist uses this interface to:

- Create exercise assignments with clinical treatment parameters (targetMinAngle, targetMaxAngle, allowedMinAngle, allowedMaxAngle, side, monitoringLevel, restrictions).
- Review patient session history including accuracy percentage, average ROM, reviewRequired flag, clinical flags, and per-repetition form breakdown.
- Manage the exercise library.

All data access uses Axios with JWT bearer-token authentication against the Spring Boot REST API.

### 5.2 Administrative CLI

The administrative CLI (admin-cli/physiosense-admin-cli/) is a standalone Spring Boot 4.1.1 CommandLineRunner application. It does not connect to PostgreSQL directly; all operations transit through authenticated REST calls via Spring RestClient. The CLI provides a console menu (ConsoleMenu.java) for interactive physiotherapist account provisioning and uses BackendClient.java as the REST API client. This design enforces the principle of least privilege: administrative operations are subject to the same business-logic validation as all other API clients.

## 6. Testing Coverage

The AI service includes a pytest test suite (ai-service/tests/) with 13 test modules verified from the directory listing:

| Module | Scope |
|---|---|
| test_geometry.py | calculate_angle_3points, calculate_angle_with_vertical, vector_2d, euclidean_distance, midpoint |
| test_state_machine.py | FSM squat rep cycle, aborted rep (not counted), RepCounter accuracy calculation |
| test_bicep_curl.py | Bicep curl analyser |
| test_shoulder_raise.py | Shoulder raise analyser |
| test_squat.py | Squat analyser |
| test_new_exercises.py | Lunge, knee extension, hamstring curl analysers |
| test_compensations.py | Compensation detection flags |
| test_clinical.py | Treatment config defaults, movement constraint validation |
| test_dynamic_thresholds.py | FSM threshold parameterisation from TreatmentConfig |
| test_landmarks.py | Landmark normalisation and virtual landmark computation |
| test_orientation.py | Camera orientation classification |
| test_api.py | FastAPI REST and WebSocket endpoint integration |

The Spring Boot backend includes JUnit test classes for ExerciseService, ExerciseAssignmentService, ExerciseSessionService, PhysiotherapistService, and UserService (confirmed from surefire-reports directory). The administrative CLI includes BackendClientTests, ConsoleMenuTests, and application context tests.

## 7. Limitations and Open Questions

### 7.1 Monocular 2D Geometry

All angle calculations use only x and y image-plane coordinates. z (MediaPipe's monocular depth estimate) is preserved in the landmark dictionary but is not used in any geometric computation (confirmed from geometry/vectors.py). Computed angles are **apparent 2D projection angles**, not true 3D anatomical joint angles. Projection error grows with the sine of the out-of-plane rotation angle. The camera orientation module mitigates this by detecting misalignment, but cannot eliminate projection error for non-ideal positions.

### 7.2 Pose Model Choice

The Lite model is used throughout. The Full and Heavy variants offer higher landmark accuracy at the cost of inference latency. For a server-side deployment not constrained by mobile hardware, the Full or Heavy model may be preferable — this is a single-line configuration change in PoseEstimator._init_detector.

### 7.3 Temporal Filtering at the MediaPipe Level

IMAGE mode bypasses MediaPipe's internal temporal filter. The application-level EMA ($\alpha$ = 0.65) is the only temporal regularisation. The optimal value of $\alpha$ has not been benchmarked against clinical angle measurements.

### 7.4 Threshold Calibration

All FSM thresholds, phase boundaries, and compensation flag thresholds are programmatic engineering constants. The 48° trunk lean limit, the 0.65 valgus ratio, and the 18° sway limit lack published clinical normative references. Prospective validation against expert-annotated movement data is required.

### 7.5 Absence of Temporal Form Metrics

Analysers process each frame independently. Angular velocity, jerk, and movement smoothness (e.g., Spectral Arc Length) are not computed. These metrics are clinically meaningful for assessing motor control quality and could be integrated at low additional computational cost.

### 7.6 Security Configuration Issues

- AI service CORS: allow_origins=["*"] (appropriate for development, not production).
- WebSocket: no JWT authentication; patient ID is sent as plain text.

- Backend JWT: expiration-ms=0 means tokens do not expire.

These issues should be resolved before clinical deployment.

### 7.7 Clinical Validation Absent

No prospective user study, RCT, or comparison against gold-standard goniometry or motion capture has been conducted. The platform should be treated as a research prototype requiring formal ethical approval and clinical evaluation before patient-facing use.

## 8. Research Contributions and Future Work

### 8.1 Summary of Contributions

1. **Treatment-parameterised AI pipeline:** clinician-issued ROM boundaries dynamically reconfigure FSM thresholds and frame-level validation at session initialisation.
2. **Six-exercise coverage** with individually configured side-selection logic and exercise-specific compensation detection.
3. **Six-state FSM with dual-mode logic** handling both flexion and elevation exercises, with hysteresis, peak verification, and minimum duration guards.
4. **Structured clinical session summary** with 11 distinct clinical flags, per-repetition form records, and a reviewRequired flag.
5. **Full-stack integration architecture** with all source files documented and communication interfaces verified.

### 8.2 Future Directions

**Engineering improvements:**

- Migrate to VIDEO running mode to engage MediaPipe temporal tracking.
- Evaluate Full/Heavy model for accuracy–latency trade-off.
- Implement JWT-authenticated WebSocket sessions.
- Enforce CORS allow-list and JWT expiry in production profiles.
- Normalise repHistory and clinicalFlags as relational columns.

**Research requiring prospective validation:**

- **Threshold optimisation** against expert-annotated exercise video.
- **Clinical accuracy study** comparing angle estimates against goniometry or motion capture.
- **Patient outcome RCT** measuring adherence and ROM recovery.
- **3D landmark integration** using MediaPipe z or stereo cameras to reduce projection error.
- **Temporal quality metrics** (SPARC, Jerk Ratio) as supplementary form indicators.
- **Personalised threshold calibration** from an initial supervised reference session.

## 9. Conclusion

PhysioSense demonstrates a technically coherent and architecturally complete approach to remote physiotherapy monitoring that bridges the gap between clinician prescription and unsupervised home exercise execution. The system's principal engineering contribution is the propagation of clinician-issued treatment parameters from a relational clinical record through a REST API and WebSocket protocol to a real-time AI inference engine that validates each exercise frame against those patient-specific boundaries. This closed clinical loop — prescribe, monitor, summarise, review — is implemented across five interoperable subsystems and verified from source-code inspection.

The AI pipeline uses the MediaPipe PoseLandmarker Lite model, 2D vector-based joint angle computation, a six-state hysteresis FSM, and a three-flag geometric compensation detector. All components are fully deterministic and interpretable. Session summaries include 11 clinical flags, per-repetition form records, and a physiotherapist-review indicator.

The system is not a validated clinical device. Thresholds require prospective optimisation, angle estimates are 2D projections subject to view-dependent error, and no controlled trial data are available. With these caveats properly understood, PhysioSense represents a functionally complete open-architecture platform on which rigorous clinical evaluation can be built.

## References

1. Bandura, A. (1997). *Self-efficacy: The exercise of control*. Freeman.
2. Bazarevsky, V., Grishchenko, I., Raveendran, K., Zhu, T., Zhang, F., and Grundmann, M. (2020). BlazePose: On-device real-time body pose tracking. *CVPR Workshop on Computer Vision for Augmented and Virtual Reality*. arXiv:2006.10204.
3. Cao, Z., Simon, T., Wei, S.-E., and Sheikh, Y. (2017). Realtime multi-person 2D pose estimation using part affinity fields. *Proceedings of the IEEE CVPR*, 7291–7299.
4. Colyer, S. L., Evans, M., Cosker, D. P., and Salo, A. I. T. (2018). A review of the evolution of vision-based motion analysis and the integration of advanced computer vision methods towards developing a markerless system. *Sports Medicine – Open*, 4(1), 24.
5. Cottrell, M. A., Galea, O. A., O'Leary, S. P., Hill, A. J., and Russell, T. G. (2017). Real-time telerehabilitation for the treatment of musculoskeletal conditions is effective and comparable to standard practice. *Clinical Rehabilitation*, 31(5), 625–638.
6. Jack, K., McLean, S. M., Moffett, J. K., and Gardiner, E. (2010). Barriers to treatment adherence in physiotherapy outpatient clinics: a systematic review. *Manual Therapy*, 15(3), 220–228.
7. Khurana, A., Sehgal, S., and Harit, G. (2023). BlazePose-based gait analysis for clinical rehabilitation monitoring. *Proceedings of the International Conference on Advances in Computing and Communication*, Springer.
8. Lea, C., Reiter, A., Vidal, R., and Hager, G. D. (2016). Segmental spatiotemporal CNNs for fine-grained action segmentation. *Proceedings of the ECCV*, Springer, 36–52.
9. Lugaresi, C., et al. (2019). MediaPipe: A framework for building perception pipelines. *arXiv preprint* arXiv:1906.08172.
10. Maddison, R., et al. (2019). Effects and acceptability of mobile health (mHealth) cardiac rehabilitation exercise intervention: randomised controlled trial. *BMJ Open*, 9(1), e021749.
11. Peek, A. L., Miller, C., and Perdomo, M. (2020). Stay active, get better: a pilot study on smartphone coaching in musculoskeletal physiotherapy patients. *Physiotherapy*, 107, 219–227.
12. Shortliffe, E. H. and Cimino, J. J. (eds.) (2014). *Biomedical informatics: Computer applications in health care and biomedicine* (4th ed.). Springer.
13. Shotton, J., et al. (2011). Real-time human pose recognition in part